

Deep Learning Avancé & Applications

Détection d'objets avec YOLO

Master Bac+5 – Data Analytics & Big Data
Mouaad AGOURRAM

2025–2026

Golden Collar

Introduction & Définitions

Métriques de la détection

Approches historiques

Architecture YOLO

Entraînement

Inférence & temps réel

Datasets & applications

Synthèse

Introduction & Définitions

Objectifs pédagogiques de cette séance

Objectif principal

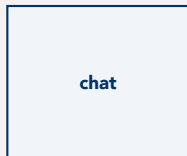
Comprendre les **concepts fondamentaux** de la détection d'objets, maîtriser l'architecture **YOLO** et savoir l'appliquer en temps réel.

À l'issue de cette séance, vous serez capable de :

- Distinguer **classification**, **localisation** et **détection**
- Comprendre les métriques spécifiques : **IoU**, **NMS**, **mAP**
- Expliquer l'architecture d'un détecteur single-shot (**YOLO**)
- Utiliser un modèle pré-entraîné pour de l'**inférence temps réel**
- Choisir la bonne variante du modèle selon la cible matérielle

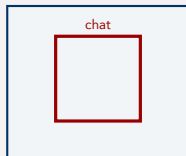
Classification vs Localisation vs Détection

Classification



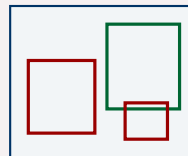
Une seule étiquette globale pour toute l'image.

Localisation



Une classe + sa bounding box (un seul objet).

Détection



Plusieurs objets, chacun avec sa classe et sa box.

⇒ La détection répond à : **"quels objets sont présents, et où ?"**

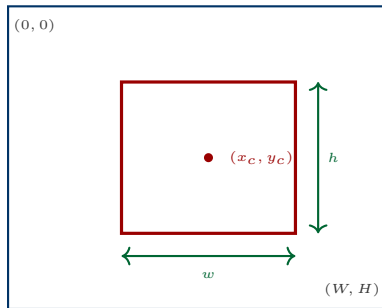
Qu'est-ce qu'une bounding box ?

Représentations courantes

- **xyxy** : (x_1, y_1, x_2, y_2)
coins haut-gauche / bas-droit
- **xywh** : (x_c, y_c, w, h)
centre + dimensions
- **normalisée** : valeurs $\in [0, 1]$
(divisées par W et H)

Annotation YOLO

Une ligne par objet : `class_id x_c y_c w h`
(normalisés)



Une détection = **(classe, bounding box, score de confiance)**.

Métriques de la détection

Intersection over Union (IoU)

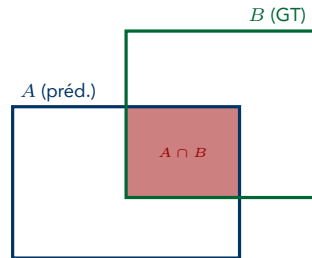
Définition

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- Mesure le **chevauchement** entre deux boxes
- Valeur $\in [0, 1]$: 0 = disjointes, 1 = identiques
- Seuil typique : **IoU** ≥ 0.5 pour valider une détection

Usage

- Évaluation : prédite vs ground truth
- Dans le NMS : supprimer les doublons



$$\text{aire}(A) + \text{aire}(B) - \text{aire}(A \cap B) = \text{aire}(A \cup B)$$

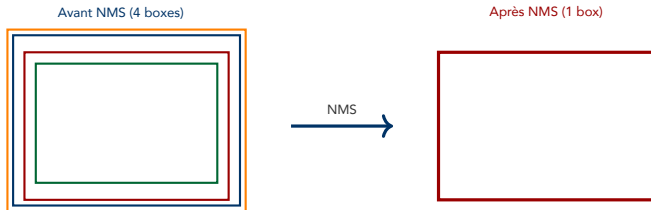
Non-Maximum Suppression (NMS)

Problème

Le modèle prédit **plusieurs boxes qui se chevauchent** pour un même objet. Il faut n'en garder qu'une.

Algorithme

1. Trier les boxes par score de confiance décroissant
2. Prendre la box avec le plus haut score → la **garder**
3. Supprimer toutes les autres boxes avec $\text{IoU} > \tau$ avec celle-ci
4. Répéter jusqu'à épuisement de la liste



Vrai / Faux positifs en détection

- **TP** : détection avec $\text{IoU} \geq 0.5$ sur un objet ground truth
- **FP** : détection sans correspondance ($\text{IoU} < 0.5$)
- **FN** : objet ground truth non détecté

Précision et Rappel

$$P = \frac{TP}{TP + FP} \quad R = \frac{TP}{TP + FN}$$

Average Precision / mAP

$$AP = \int_0^1 P(R) dR$$

$$mAP = \frac{1}{C} \sum_{c=1}^C AP_c$$

Seuils classiques

- **mAP@0.5** : AP avec un seul seuil d' $\text{IoU} = 0.5$ (métrique Pascal VOC)
- **mAP@0.5:0.95** : moyenne sur 10 seuils d' IoU (0.5 à 0.95, pas 0.05) — **métrique COCO**, plus stricte

Approches historiques

Two-stage detectors

1. **Region proposals** : générer des régions candidates
2. **Classification** de chaque région

Exemples : R-CNN (2014), Fast R-CNN, Faster R-CNN, Mask R-CNN

- + Très précis
- Lent, **non temps réel**

One-stage / single-shot

Une seule passe forward produit directement les **boxes + classes + scores**.

Exemples : **YOLO** (v1 à v11), SSD, RetinaNet

- + Rapide, **temps réel**
- Historiquement moins précis (l'écart se réduit fortement)

⇒ Pour du **temps réel** (webcam, vidéo, edge), on choisit un détecteur single-shot.

La famille R-CNN : trois générations

R-CNN (2014)

1. *Selective Search* → ~ 2000 propositions
2. CNN sur **chaque** crop
3. SVM + régression de box

– ~ 47 s / image

Fast R-CNN (2015)

1. CNN **une seule fois** sur l'image
2. **Rol Pooling** sur la feature map
3. Softmax + régression joints

+ ~ 0.3 s / image

Faster R-CNN (2015)

1. Backbone partagé
2. **RPN** (Region Proposal Network) appris
3. End-to-end

+ ~ 5 FPS

Mask R-CNN (He et al., 2017)

Extension de Faster R-CNN avec une branche de **segmentation d'instances**. Introduit le **Rol Align** (pas de quantisation spatiale), indispensable pour des masques précis au pixel près.

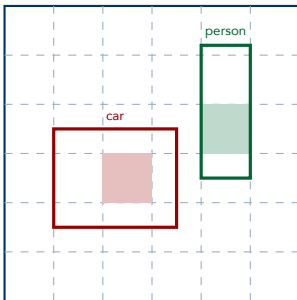
Architecture YOLO

L'idée centrale : You Only Look Once

Intuition

Au lieu de balayer l'image avec des propositions de régions, YOLO **découpe l'image en une grille $S \times S$** .
Chaque cellule prédit :

- B bounding boxes (coordonnées + score de confiance)
- Les probabilités de classe $P(\text{classe} \mid \text{objet})$



Chaque objet est "affecté" à la **cellule contenant son centre**.

Sortie d'une cellule (YOLOv1)

Tenseur de sortie : $S \times S \times (B \cdot 5 + C)$

- $S \times S$: taille de la grille
- B boxes par cellule, chacune donne $(x, y, w, h, \text{conf})$ — 5 valeurs
- C scores de classe par cellule (partagés entre les B boxes)

Exemple YOLOv1 : $S = 7$, $B = 2$, $C = 20$ (VOC)

Sortie = $7 \times 7 \times 30$ — **une seule passe forward** du CNN suffit.

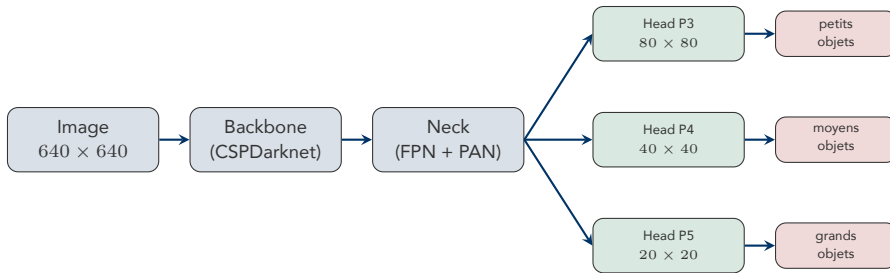
Score de confiance

$$\text{conf} = P(\text{objet}) \times \text{IoU}_{\text{pred}}^{\text{truth}}$$

→ renseigne à la fois sur la **présence** d'un objet et sur la **qualité** de la box.

Architecture globale moderne (YOLOv3+)

De la grille unique de YOLOv1, les YOLO modernes évoluent vers une architecture **Backbone / Neck / trois Heads** prédisant simultanément à trois échelles.



Rôle de chaque bloc

- **Backbone** : extraction de features multi-échelle (CNN profond)
- **Neck** : fusion des features à différentes résolutions (FPN / PAN)
- **Heads** : **trois branches de prédiction** (P3 / P4 / P5), une par échelle

Multi-échelle : pourquoi trois têtes ?

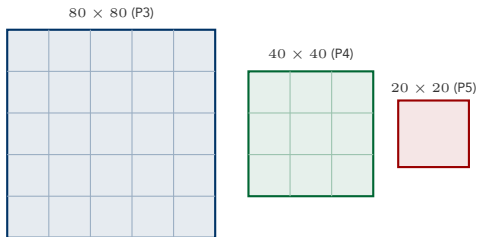
Problème

Un CNN naïf détecte mal les **petits objets**. Les features profondes ont une grande réceptivité mais une **faible résolution spatiale**.

Solution : Feature Pyramid Network

Prédire à **3 échelles différentes** (entrée 640×640) :

- 80×80 (P3) → petits objets
- 40×40 (P4) → objets moyens
- 20×20 (P5) → grands objets



Historique : ce mécanisme a été introduit dans **YOLOv3** (2018) avec les FPN. Il est conservé dans YOLOv8+ anchor-free : les trois branches restent, mais elles prédisent directement les coordonnées sans ancres de référence.

Anchor-based (YOLOv3–v5)

- Chaque cellule prédit un **delta** par rapport à plusieurs **ancres** prédéfinies
- Ancres calculées par clustering (k-means) sur le dataset
- Permet de couvrir des ratios variés
- – hyperparamètres sensibles au dataset

Anchor-free (YOLOv8+)

- Prédiction **directe** des coordonnées
- Moins d'hyperparamètres à régler
- Meilleure généralisation sur datasets atypiques
- Code et entraînement plus simples

Tendance actuelle

Les architectures modernes (YOLOv8, YOLOv11) abandonnent les ancres pour une approche **direct regression** plus simple et plus performante.

Anchor-based (YOLOv3–v5) : 3 termes

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{obj}} \mathcal{L}_{\text{obj}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}$$

- $\mathcal{L}_{\text{box}}$: régression des coordonnées (CloU / DIoU)
- $\mathcal{L}_{\text{obj}}$: présence d'un objet (BCE sur le score d'objectness)
- $\mathcal{L}_{\text{cls}}$: classe de l'objet (BCE / Focal Loss)

Anchor-free (YOLOv8+) : 2 termes

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}$$

- $\mathcal{L}_{\text{box}}$: CloU + **DFL** (régression de distribution)
- **Pas de $\mathcal{L}_{\text{obj}}$** : sans ancrs, pas de score d'objectness séparé
- $\mathcal{L}_{\text{cls}}$: BCE avec Focal Loss

Rappel Séance 1

La **Focal Loss** est utilisée pour $\mathcal{L}_{\text{cls}}$ car le fond est fortement majoritaire face aux objets. Formules détaillées : slide suivant.

CloU loss (Zheng et al., AAAI 2020)

$$\mathcal{L}_{\text{CloU}} = 1 - \text{IoU} + \underbrace{\frac{\rho^2(b, b^{gt})}{c^2}}_{\text{distance des centres}} + \underbrace{\alpha v}_{\text{ratio d'aspect}}$$

avec

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h} \right)^2, \quad \alpha = \frac{v}{(1 - \text{IoU}) + v}$$

et c la diagonale de l'enveloppe englobant les deux boxes.

CloU corrige trois défauts de l'IoU nue : **gradient nul** si non-chevauchement, **centres éloignés** non pénalisés, **mauvais ratio** non pénalisé.

Focal Loss (Lin et al., ICCV 2017)

$$\mathcal{L}_{\text{focal}} = -\alpha_t (1 - p_t)^\gamma \log(p_t), \quad \gamma \approx 2$$

Le facteur $(1 - p_t)^\gamma$ **écrase** la contribution des exemples faciles (fond majoritaire) et concentre le gradient sur les **positifs difficiles**.

Évolution des versions YOLO

Version	Année	Apports principaux
YOLOv1	2015/16	Grille $S \times S$, single-shot, ~ 45 FPS
YOLOv2/v3	2017–18	Ancres, multi-échelle (FPN), Darknet-53
YOLOv4	2020	CSPNet, Mosaic aug, Mish, CloU loss
YOLOv5	2020	Ultralytics, PyTorch natif, export facile
YOLOv7	2022	E-ELAN, optimisations inférence
YOLOv8	2023	Anchor-free , architecture simplifiée
YOLOv9	2024	PGI, architecture GELAN
YOLOv10	2024	NMS-free (end-to-end), dual assignment
YOLOv11	2024	C3k2, C2PSA (attention), SPPF, tête découplée

Variantes de taille

n (nano) < s < m < l < x — trade-off **vitesse vs précision**. En TP : yolov8n.pt (6 Mo, >100 FPS).

Tendance 2024 : la fin du NMS

YOLOv10 et les **DETR** suppriment le NMS → détection **end-to-end** (cf. slides NMS et Transformer).

DETR (Carion et al., ECCV 2020)

Premier détecteur **end-to-end**, sans ancres **ni** NMS. Pipeline :

CNN backbone $\rightarrow$ **Transformer encoder-decoder** $\rightarrow N$ prédictions en parallèle

Idées clés

- **Object queries** apprises : $N \approx 100$ vecteurs qui “interrogent” la scène
- **Hungarian matching** : appariement bijectif prédictions $\leftrightarrow$ GT pendant le train
- Pas de NMS : l’unicité est imposée *par construction* via l’appariement bijectif
- Loss : combinaison de **classification** + **L1** / **GloU** sur les boxes

Descendance

Deformable-DETR (2021, attention locale), **DINO** (2023, denoising query), **RT-DETR** (2024). RT-DETR atteint la vitesse d’un YOLOv8 tout en surpassant sa mAP sur COCO.

Entraînement

Structure attendue

```
1 dataset/  
2   images/  
3     train/  img1.jpg, img2.jpg, ...  
4     val/    img3.jpg, ...  
5   labels/  
6     train/  img1.txt, img2.txt, ...  
7     val/    img3.txt, ...  
8   data.yaml
```

Un fichier .txt par image

Une ligne par objet :

```
1 class_id  x_center  y_center  width  height
```

Toutes les valeurs sont **normalisées** entre 0 et 1.

data.yaml

Décrit les chemins train/val, le nombre de classes nc et leurs names.

Augmentations classiques

- Flip horizontal
- Crops / zooms aléatoires
- HSV jitter (teinte, saturation, luminosité)
- Rotation, translation

Spécifiques YOLO

- **Mosaic** : assemble 4 images en une
- **MixUp** : combinaison pondérée de 2 images
- Copy-paste : insertion d'objets

Pourquoi Mosaic ?

Améliore la détection de **petits objets** et expose le modèle à plus de contextes par batch → meilleure généralisation.

Label assignment : quel objet pour quelle prédiction ?

Le problème

Pour chaque objet du ground truth, **quelle(s) prédiction(s)** doit-on rendre responsables de cet objet ? Un mauvais appariement déséquilibre la loss et dégrade fortement la mAP.

Assignment statique (YOLOv3 / v4)

- Une GT → **une seule** ancre (IoU max)
- Indépendant de la prédiction courante
- Très déséquilibré positifs / négatifs
- Rigide : dépend uniquement de la géométrie

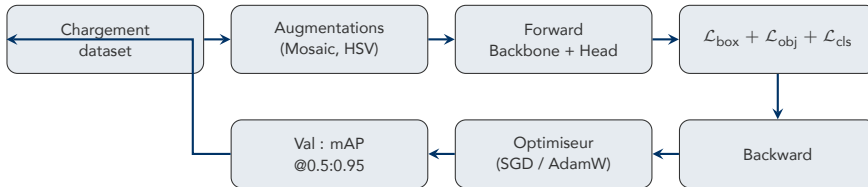
Assignment dynamique (v6 / v8 / v10)

- **SimOTA** (YOLOX, 2021) : vu comme un problème de *transport optimal*
- **TAL** (Task-Aligned Learning) : score $t = s^\alpha \cdot \text{IoU}^\beta$
- Dépend de la **qualité courante** des prédictions
- Top- k candidats par GT

Pourquoi c'est essentiel

L'amélioration majeure des YOLO récents ne vient pas tant du backbone que du **label assignment dynamique** : typiquement +2 à +4 points de mAP pour un coût d'inférence **nul**.

Pipeline d'entraînement

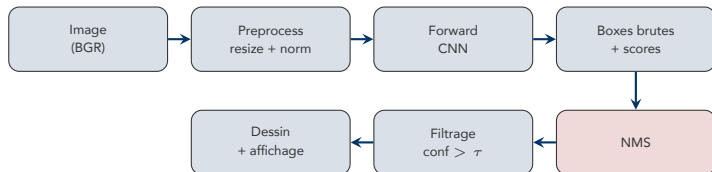


En pratique : transfer learning

On part d'un modèle pré-entraîné sur **COCO**, puis fine-tuning sur le dataset cible pour quelques dizaines d'epochs. Beaucoup plus rapide et efficace qu'un entraînement *from scratch*.

Inférence & temps réel

Pipeline d'inférence



En pratique avec Ultralytics

```
1 from ultralytics import YOLO
2 model = YOLO("yolov8n.pt")
3 results = model(frame, conf=0.4, iou=0.5)
```

Le NMS et le filtrage sont **intégrés** dans l'API — on ne manipule que les résultats finaux.

Modèle	Params (M)	mAP@0.5:0.95	mAP@0.5	Latence (ms)
Faster R-CNN R50-FPN	42	40.2	61.0	~ 40
RetinaNet R101	53	40.8	61.1	~ 35
YOLOv5-s	7.2	37.4	56.8	2.0
YOLOv5-x	86.7	50.7	68.9	6.1
YOLOv8-n	3.2	37.3	52.9	1.5
YOLOv8-s	11.2	44.9	61.8	2.4
YOLOv8-x	68.2	53.9	71.0	7.5
DETR-DC5 R101	60	44.9	64.7	~ 28
RT-DETR-R50	42	53.1	71.3	9.2

Lecture

Chiffres issus des papiers originaux et des *release notes* Ultralytics (latences en FP16 sur GPU A100 640×640). À taille comparable, les YOLO modernes et RT-DETR **dominent** les two-stage sur le trade-off précision / vitesse.

Modèle	Paramètres	FPS (GPU)	mAP@0.5:0.95 COCO
YOLOv8n	3.2 M	> 250	37.3
YOLOv8s	11.2 M	~ 170	44.9
YOLOv8m	25.9 M	~ 100	50.2
YOLOv8l	43.7 M	~ 70	52.9
YOLOv8x	68.2 M	~ 45	53.9

Pour du temps réel sur CPU ou edge

- Utiliser **yolov8n** ou **yolov8s**
- Réduire `imgsz` (par ex. 416 au lieu de 640)
- Exporter en **ONNX** / **TFLite** / **OpenVINO** selon la cible matérielle
- Quantisation **INT8** pour un gain de vitesse $\times 2$ – $\times 4$

Datasets & applications

COCO

- ~ 330k images
- **80 classes**
- Objets du quotidien (personne, voiture, animaux, meubles...)
- **Benchmark standard**

Pascal VOC

- ~ 11k images
- 20 classes
- Historique, encore utile pour des tests rapides

Open Images

- ~ 9M images
- 600 classes
- Très large mais plus bruité

Datasets personnalisés

Outils d'annotation : LabelImg, Roboflow, CVAT. Pour fine-tuner un YOLO sur un nouveau cas d'usage, il suffit généralement de **quelques centaines d'images annotées**.

Industrie

- Contrôle qualité (défauts visuels)
- Comptage en entrepôt / logistique
- Sécurité : détection d'EPI (casques, gilets)
- Lecture de codes-barres / plaques

Mobilité

- Conduite autonome (piétons, panneaux)
- ADAS (assistance conducteur)
- Surveillance du trafic urbain

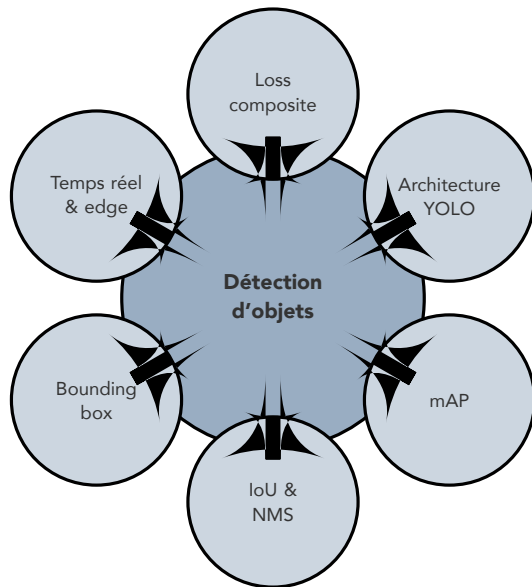
Santé

- Détection de lésions en imagerie médicale
- Analyse automatique de cellules

Grand public

- Filtres AR (Snapchat, Instagram)
- Reconnaissance de produits (e-commerce)
- Smart-home (détection personnes / animaux)

Synthèse



Les essentiels

- Détection = classification + **localisation multiple**
- Sortie par objet : (class, box, confidence)
- **IoU** mesure le chevauchement, **NMS** supprime les doublons
- **mAP** est LA métrique d'évaluation de référence
- YOLO = single-shot $\Rightarrow$ **temps réel**
- Loss composite : **box + obj + cls**
- Pour déployer : choisir la taille du modèle en fonction de la cible matérielle

Objectifs du TP (3h)

- Écrire 4 fonctions utilitaires pures :
`filter_detections`, `count_by_class`, `compute_iou`, `draw_detections`
- Valider chaque fonction avec **pytest**
- Lancer YOLOv8n sur sa webcam en temps réel
- Implémenter un compteur dans une zone d'intérêt (ROI)
- **Bonus** : utiliser son téléphone comme caméra IP

Rendu

Notebook exécuté + tests pytest au vert + 2-3 captures d'écran de la détection en action.

Questions ?

Place au TP : *YOLO temps réel sur votre webcam*